

## Introducción

La transformación digital ha generado nuevas oportunidades para optimizar procesos en distintos sectores, incluyendo el área agrícola, donde muchas comunidades todavía enfrentan dificultades para promocionar y comercializar sus productos de manera eficiente. En numerosas ocasiones, los productores rurales dependen únicamente de recomendaciones personales, mercados locales o intermediarios, lo que limita el alcance de sus productos y reduce las posibilidades de generar mayores oportunidades comerciales.

Con el propósito de aportar una solución tecnológica accesible y funcional, se desarrolló “Cosecha Red”, una plataforma digital orientada a facilitar la interacción entre productores y compradores dentro de la comunidad agrícola rural. El sistema funciona como un catálogo digital de productos agrícolas, permitiendo que los productores publiquen información sobre sus productos y que los compradores puedan consultar la disponibilidad existente y contactar directamente a los productores para establecer negociaciones.

El proyecto fue desarrollado utilizando una arquitectura web moderna basada en React y Tailwind CSS para la construcción de la interfaz de usuario, FastAPI con Python para la lógica del backend y PostgreSQL como sistema gestor de base de datos. Asimismo, la comunicación entre los diferentes componentes se realiza mediante una API REST, permitiendo mantener una estructura organizada, escalable y desacoplada.

La presente documentación corresponde a la tercera fase del proyecto y presenta la primera versión funcional del sistema, identificada como v1.0.0. En este documento se describen los aspectos técnicos y funcionales más importantes de la aplicación, incluyendo requerimientos, arquitectura, estructura del sistema, control de versiones, configuración del entorno, trazabilidad, pruebas y funcionalidades implementadas durante el desarrollo de esta versión inicial.

## Objetivo General

Desarrollar una plataforma web denominada “Cosecha Red” que permita facilitar la comercialización de productos agrícolas dentro de la comunidad rural “La Esperanza”, mediante un sistema digital que conecte a productores y compradores a través de un catálogo de publicaciones, registro de interacciones y gestión de usuarios, utilizando tecnologías web modernas para garantizar una solución funcional, organizada y accesible.

## Objetivos Específicos

- Diseñar una plataforma web que permita a los productores agrícolas publicar sus productos de manera organizada y accesible.
- Implementar un sistema de autenticación y control de acceso basado en roles para diferenciar las funcionalidades de productores, compradores y administradores.
- Desarrollar un catálogo digital que facilite a los compradores consultar productos disponibles y filtrar información según sus necesidades.
- Permitir el registro de interacciones entre productores y compradores para mantener un historial y trazabilidad dentro del sistema.
- Implementar un sistema de valoraciones y reputación que contribuya a generar mayor confianza entre los usuarios de la plataforma.
- Utilizar tecnologías web modernas como React, FastAPI y PostgreSQL para desarrollar una solución escalable, estructurada y funcional.
- Aplicar buenas prácticas de desarrollo de software, control de versiones y documentación técnica durante la construcción del proyecto.

## Alcance del Release v1.0.0

La versión v1.0.0 de Cosecha Red corresponde a la primera versión funcional del sistema, enfocada en implementar las funcionalidades principales necesarias para permitir la interacción entre productores y compradores dentro de la comunidad agrícola rural.

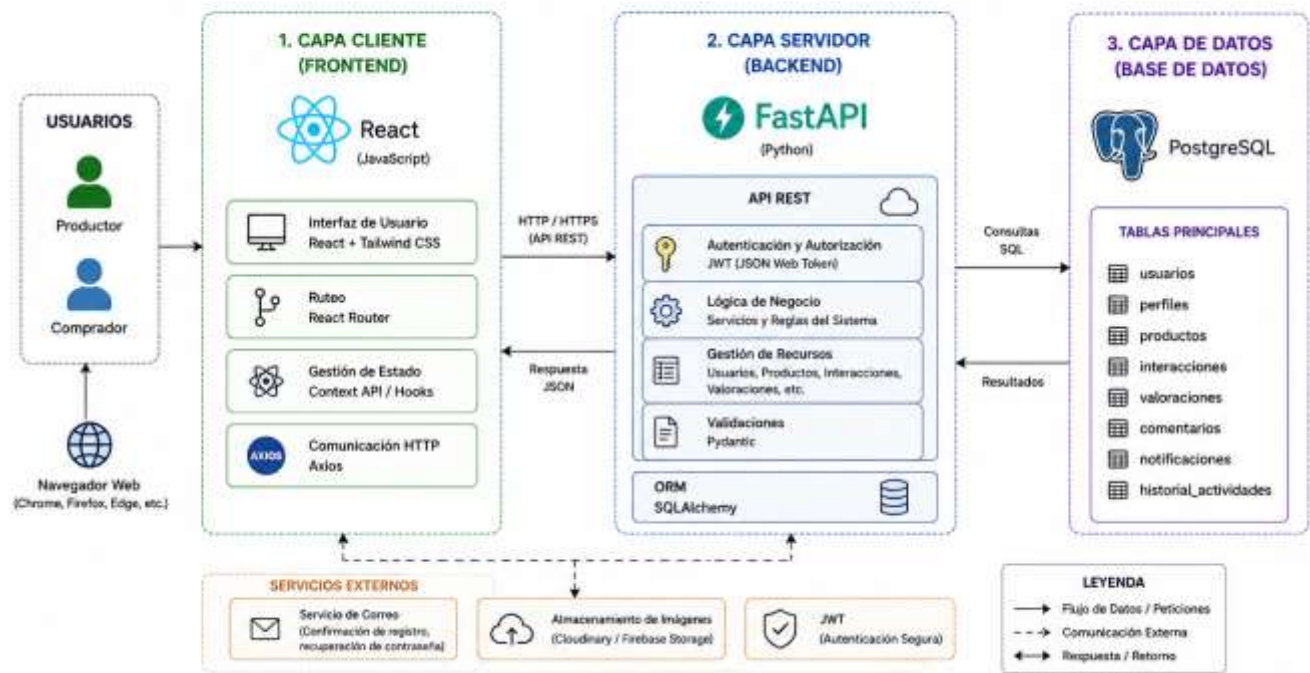
Esta versión incluye módulos de registro e inicio de sesión de usuarios, gestión de perfiles, publicación y administración de productos agrícolas, consulta del catálogo de productos, registro de interacciones entre usuarios, historial de actividades, sistema de valoraciones y un panel básico de administración. Asimismo, se implementó control de acceso por roles y autenticación mediante tokens JWT para garantizar una gestión más segura de la información.

El sistema está diseñado como una plataforma web accesible desde navegadores modernos y utiliza una arquitectura basada en API REST para la comunicación entre frontend, backend y base de datos.

Dentro de esta primera versión no se contempla la implementación de pagos en línea, sistema de notificaciones, recuperación de contraseña vía correo electrónico ni despliegue en infraestructura cloud, ya que estas funcionalidades se consideran mejoras futuras para próximas versiones del proyecto.

# DIAGRAMA DE ARQUITECTURA – COSECHA RED

Arquitectura Cliente – Servidor de tres capas



El sistema Cosecha Red está basado en una arquitectura de tres capas que separa claramente el frontend, backend y base de datos. El frontend está desarrollado en React y se ejecuta en el navegador del usuario, encargado de la interfaz y la interacción.

El backend está implementado en FastAPI (Python) y gestiona la lógica del sistema, autenticación, validaciones y el procesamiento de las solicitudes mediante una API REST. La base de datos utiliza PostgreSQL, donde se almacena toda la información del sistema como usuarios, productos, interacciones y valoraciones.

La comunicación entre las capas se realiza mediante peticiones HTTP, lo que permite una arquitectura desacoplada, escalable y fácil de mantener.

## Descripción de componentes

El sistema “Cosecha Red” está compuesto por tres componentes principales que trabajan de forma integrada para garantizar el funcionamiento completo de la plataforma.

### 1. Capa de Cliente (Frontend)

Corresponde a la interfaz de usuario desarrollada en React con Tailwind CSS. Es el componente encargado de mostrar la información al usuario y permitir la interacción con el sistema mediante formularios, navegación y consumo de datos desde la API. Se ejecuta directamente en el navegador.

### 2. Capa de Servidor (Backend)

Está implementada en FastAPI utilizando Python. Es el núcleo del sistema, ya que procesa la lógica de negocio, gestiona las solicitudes del cliente y controla el acceso a los recursos. También se encarga de la autenticación de usuarios mediante JWT, la validación de datos y la comunicación con la base de datos.

### 3. Capa de Datos (Base de Datos)

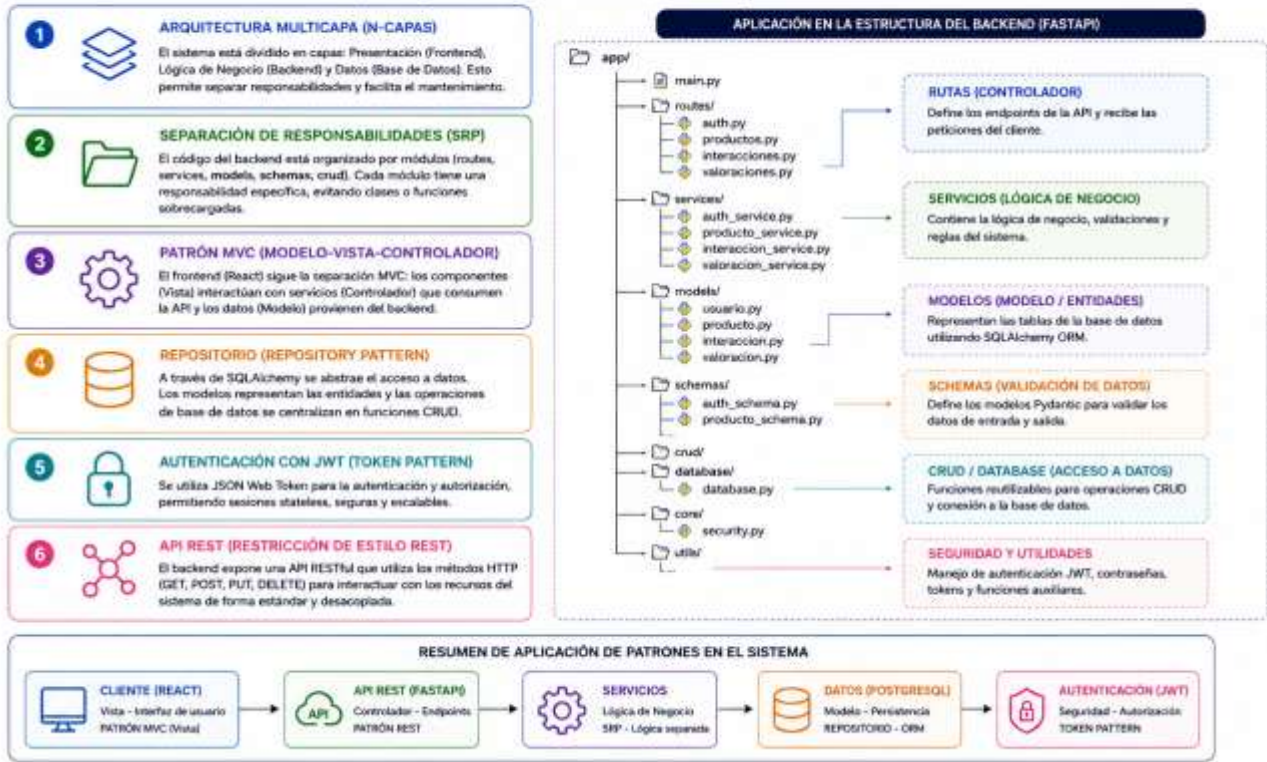
Utiliza PostgreSQL como sistema gestor de base de datos. Aquí se almacena toda la información del sistema, incluyendo usuarios, productos, interacciones y valoraciones. Garantiza la persistencia, integridad y consistencia de los datos.

### 4. Servicios externos (apoyo del sistema)

El sistema puede integrarse con servicios externos como correo electrónico para notificaciones y recuperación de cuenta, así como servicios de almacenamiento de archivos si se requieren en futuras versiones.

## 14. PATRONES DE DISEÑO APLICADOS

En el desarrollo del sistema Cosecha Red se aplicaron diversos patrones de diseño y buenas prácticas para lograr un código mantenible, escalable y fácil de extender.



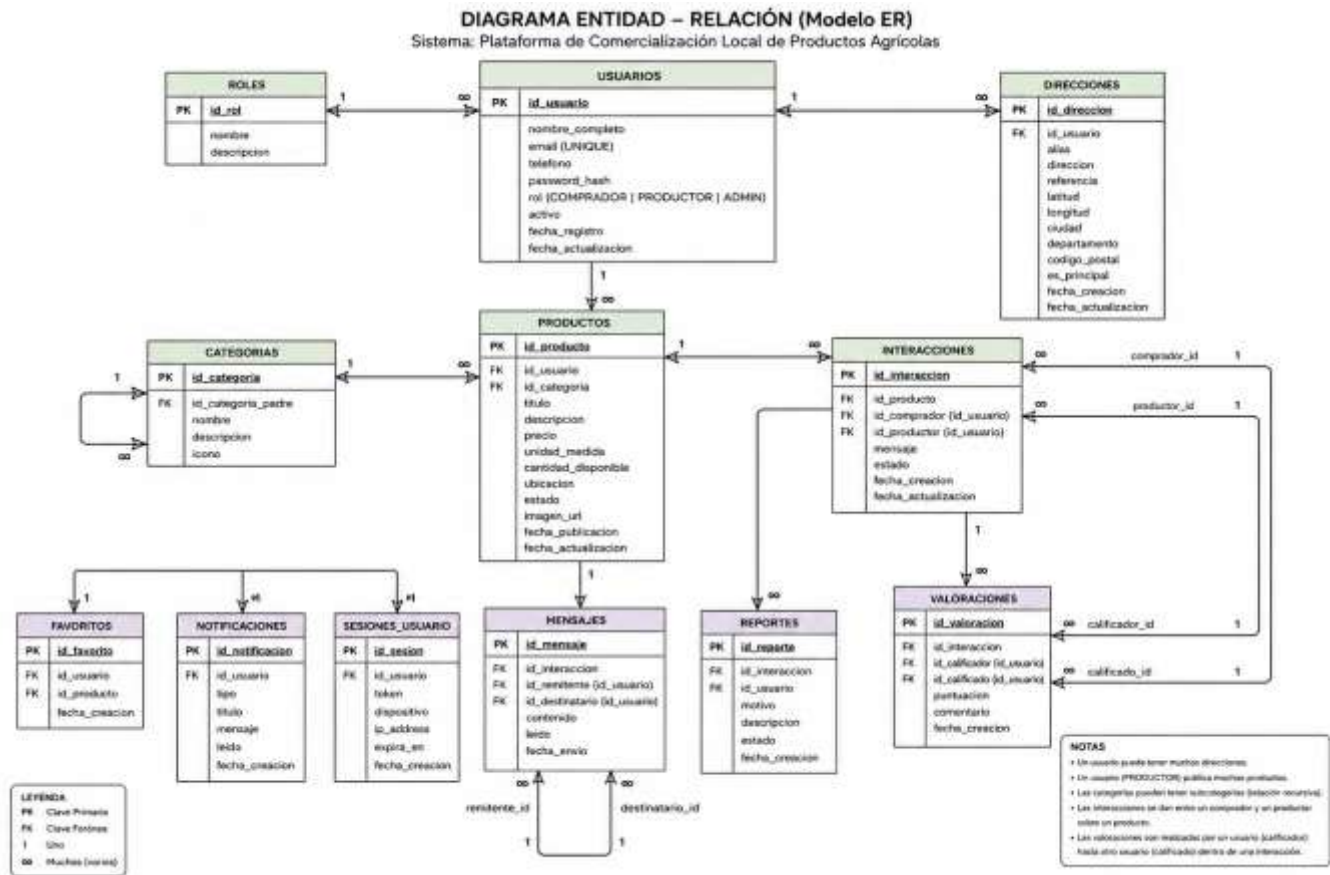
El sistema “Cosecha Red” implementa una arquitectura basada en buenas prácticas de desarrollo de software, con el objetivo de mejorar la organización del código, su mantenibilidad y escalabilidad. Se utiliza una arquitectura multicapa, donde el sistema se divide en tres niveles principales: frontend, backend y base de datos. Esta separación permite aislar responsabilidades y facilita el mantenimiento del sistema.

En el backend se aplica el principio de separación de responsabilidades (SRP), organizando el código en módulos como rutas, servicios, modelos y esquemas, donde cada uno cumple una función específica dentro del sistema.

El frontend sigue un enfoque basado en el patrón MVC simplificado, donde la vista corresponde a los componentes de React, el modelo a la información consumida desde la API y el controlador a la lógica de interacción del usuario.

Además, se implementa el patrón repositorio mediante SQLAlchemy, lo que permite abstraer el acceso a datos y centralizar las operaciones de base de datos. El sistema también utiliza autenticación basada en JWT, lo que permite manejar sesiones seguras mediante tokens sin necesidad de almacenar estados en el servidor. Finalmente, la API sigue el estilo REST, permitiendo la comunicación entre cliente y servidor mediante métodos HTTP estándar como GET, POST, PUT y DELETE.

Modelo entidad-relación



Diccionario de datos

| Campo         | Tipo de dato | Seguridad / Descripción                             |
|---------------|--------------|-----------------------------------------------------|
| id            | Integer      | Identificador único del usuario (clave primaria)    |
| full_name     | String(150)  | Nombre completo del usuario                         |
| email         | String(150)  | Correo único del usuario (evita duplicados)         |
| phone         | String(20)   | Teléfono de contacto                                |
| password_hash | String       | Contraseña encriptada (NO se guarda en texto plano) |
| role          | Enum         | Rol del usuario: COMPRADOR, PRODUCTOR o ADMIN       |
| is_active     | Boolean      | Indica si el usuario está activo o deshabilitado    |
| created_at    | DateTime     | Fecha de creación del usuario                       |
| updated_at    | DateTime     | Fecha de última modificación                        |

## Script de creación de base de datos

```
CREATE USER cosecha_user  
WITH PASSWORD '123456';
```

```
CREATE DATABASE cosecha_red  
OWNER cosecha_user;
```

```
GRANT ALL PRIVILEGES  
ON DATABASE cosecha_red  
TO cosecha_user;
```

```
CREATE SCHEMA auth;  
CREATE SCHEMA catalog;  
CREATE SCHEMA interaction;  
CREATE SCHEMA audit;
```

```
CREATE TYPE auth.user_role AS ENUM (  
  'COMPRADOR',  
  'PRODUCTOR',  
  'ADMIN'  
);
```

```
CREATE TYPE interaction.review_status AS ENUM (  
  'PENDING',  
  'APPROVED',  
  'REJECTED'  
);
```

```
CREATE TABLE auth.users (  
  id SERIAL PRIMARY KEY,  
  
  full_name VARCHAR(150) NOT NULL,  
  
  email VARCHAR(150) UNIQUE NOT NULL,  
  
  phone VARCHAR(20) NOT NULL,  
  
  password_hash TEXT NOT NULL,  
  
  role auth.user_role NOT NULL,  
  
  is_active BOOLEAN DEFAULT TRUE,  
  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE catalog.products (  
  id SERIAL PRIMARY KEY,
```



```
producer_id INTEGER NOT NULL,  
  
title VARCHAR(150) NOT NULL,  
  
quantity NUMERIC(10,2) NOT NULL,  
  
unit VARCHAR(50) NOT NULL,  
  
price NUMERIC(10,2) NOT NULL,  
  
location VARCHAR(150) NOT NULL,  
  
description TEXT,  
  
is_active BOOLEAN DEFAULT TRUE,  
  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
CONSTRAINT fk_products_producer  
FOREIGN KEY (producer_id)  
REFERENCES auth.users(id)  
);
```

```
CREATE TABLE interaction.interactions (  
  id SERIAL PRIMARY KEY,  
  
  buyer_id INTEGER NOT NULL,  
  
  producer_id INTEGER NOT NULL,  
  
  product_id INTEGER NOT NULL,  
  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
  CONSTRAINT fk_interaction_buyer  
  FOREIGN KEY (buyer_id)  
  REFERENCES auth.users(id),  
  
  CONSTRAINT fk_interaction_producer  
  FOREIGN KEY (producer_id)  
  REFERENCES auth.users(id),  
  
  CONSTRAINT fk_interaction_product  
  FOREIGN KEY (product_id)  
  REFERENCES catalog.products(id)  
);
```

```
CREATE TABLE interaction.reviews (  
  id SERIAL PRIMARY KEY,  
  
  interaction_id INTEGER UNIQUE NOT NULL,
```

```
reviewer_id INTEGER NOT NULL,

reviewed_user_id INTEGER NOT NULL,

rating INTEGER NOT NULL
CHECK (rating BETWEEN 1 AND 5),

comment TEXT NOT NULL,

status interaction.review_status
DEFAULT 'PENDING',

rejection_reason TEXT,

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

approved_at TIMESTAMP,

CONSTRAINT fk_review_interaction
FOREIGN KEY (interaction_id)
REFERENCES interaction.interactions(id),

CONSTRAINT fk_review_reviewer
FOREIGN KEY (reviewer_id)
REFERENCES auth.users(id),

CONSTRAINT fk_reviewed_user
FOREIGN KEY (reviewed_user_id)
REFERENCES auth.users(id)
);

CREATE TABLE audit.audit_logs (
  id SERIAL PRIMARY KEY,

  admin_id INTEGER NOT NULL,

  action VARCHAR(150) NOT NULL,

  entity_type VARCHAR(100) NOT NULL,

  entity_id INTEGER NOT NULL,

  reason TEXT,

  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  CONSTRAINT fk_audit_admin
  FOREIGN KEY (admin_id)
  REFERENCES auth.users(id)
);
```

## Documentación de APIs

El sistema “Cosecha Red” expone una API REST desarrollada con FastAPI, la cual permite la comunicación entre el frontend (React) y el backend del sistema. Esta API gestiona las operaciones principales del sistema, incluyendo autenticación de usuarios y administración de información.

La API utiliza el protocolo HTTP y responde en formato JSON, lo que permite una integración sencilla y estructurada entre los diferentes componentes del sistema.

- **POST /auth/register**

Permite registrar un nuevo usuario en el sistema.

**Descripción:** Crea una cuenta de usuario con rol asignado.

**Respuesta exitosa (201):**

```
{
  "message": "Usuario creado correctamente"
}
```

- **POST /auth/login**

Permite iniciar sesión en el sistema.

**Descripción:** Valida credenciales y genera un token JWT.

**Respuesta exitosa (200):**

```
{
  "access_token": "token_generado",
  "token_type": "bearer"
}
```

- **GET /users/me**

Obtiene la información del usuario autenticado.

**Descripción:** Retorna los datos del usuario en sesión.

**Respuesta:**

```
{
  "id": 1,
  "full_name": "Juan Perez",
  "email": "correo@ejemplo.com",
  "phone": "12345678",
  "role": "PRODUCTOR",
  "is_active": true
}
```

- **PUT /users/me**

Actualiza la información del usuario autenticado.

**Descripción:** Permite modificar datos personales como nombre y teléfono.

## Ejemplos de Request / Response

### Auth – Login Request

```
{  
  "email": "usuario@ejemplo.com",  
  "password": "1234"  
}
```

### Response

```
{  
  "access_token": "jwt_token_generado",  
  "token_type": "bearer"  
}
```

### Usuario - Registro Request

```
{  
  "full_name": "Juan Perez",  
  "email": "juan@correo.com",  
  "phone": "12345678",  
  "password": "1234",  
  "role": "PRODUCTOR"  
}
```

### Response

```
{  
  "message": "Usuario creado correctamente",  
  "user_id": 1  
}
```

### Usuario - Obtener perfil Request

GET /users/me  
Authorization: Bearer <token>

### Response

```
{  
  "id": 1,  
  "full_name": "Juan Perez",  
  "email": "juan@correo.com",  
  "phone": "12345678",  
  "role": "PRODUCTOR",  
  "is_active": true  
}
```

## Códigos de error de API

Los códigos de error de la API indican el resultado de las solicitudes realizadas al sistema y permiten identificar si una operación fue exitosa o si ocurrió algún problema durante su ejecución. La API del sistema “Cosecha Red” utiliza códigos HTTP estándar para manejar las respuestas del servidor, facilitando la comunicación entre el frontend y el backend.

### 401 Unauthorized

Se produce cuando el usuario intenta acceder a un recurso sin estar autenticado o cuando el token JWT es inválido o ha expirado.

### 403 Forbidden

Indica que el usuario está autenticado, pero no tiene permisos para realizar la acción solicitada según su rol.

### 404 Not Found

Se genera cuando el recurso solicitado no existe en el sistema, por ejemplo, un usuario o registro inexistente.

### 400 Bad Request

Ocurre cuando los datos enviados por el cliente no cumplen con las validaciones requeridas o están incompletos.

### 500 Internal Server Error

Indica un error inesperado en el servidor durante el procesamiento de la solicitud.

Estos códigos son utilizados por FastAPI de forma estándar junto con la implementación de la lógica del sistema, permitiendo un control adecuado de errores y respuestas en la API REST.

## Guía de instalación corregida para FastAPI / Python

### Requisitos previos

Antes de ejecutar el proyecto, se debe contar con los siguientes componentes instalados en el sistema:

- Python 3.10 o superior
- PostgreSQL 15 o superior
- Git
- Editor de código (Visual Studio Code recomendado)

### 1. Clonar el repositorio

Se debe descargar el proyecto desde el repositorio de GitHub:

```
git clone https://github.com/egvalle/Analisis1_Proyecto.git
cd Analisis1_Proyecto
```

### 2. Acceder al backend

Entrar a la carpeta del backend:

```
cd backend
```

### 3. Crear entorno virtual

```
python -m venv venv
```

Activar entorno virtual:  
Windows:

```
venv\Scripts\activate
```

### 4. Instalar dependencias

```
pip install -r requirements.txt
```

### 5. Configurar variables de entorno

Crear un archivo .env dentro de la carpeta backend con la siguiente estructura:

```
DB_HOST=localhost  
DB_PORT=5432  
DB_USER=postgres  
DB_PASSWORD=1234  
DB_NAME=cosecha_red
```

### 6. Ejecutar el servidor

```
uvicorn app.main:app --reload
```

### 7. Acceder a la API

La API estará disponible en:

<http://127.0.0.1:8000>

Documentación automática (Swagger):  
<http://127.0.0.1:8000/docs>

## Guía de configuración del entorno de desarrollo

Para el desarrollo del sistema “Cosecha Red” se requiere configurar correctamente el entorno de trabajo, asegurando que todas las herramientas necesarias estén instaladas y funcionando de manera adecuada.

### Editor de código

Se recomienda el uso de Visual Studio Code como entorno de desarrollo principal debido a su compatibilidad con Python, JavaScript y herramientas de control de versiones.

Extensiones recomendadas

- Python
- Pylance
- ESLint
- Prettier
- GitLens

## Entorno backend (FastAPI)

El backend del sistema está desarrollado en Python utilizando FastAPI, por lo que es necesario crear un entorno virtual para manejar dependencias de forma aislada.

Creación del entorno virtual:

```
python -m venv venv
```

Activación del entorno virtual:

Windows:

```
venv\Scripts\activate
```

## Instalación de dependencias

Dentro del entorno virtual se deben instalar las librerías necesarias:

```
pip install -r requirements.txt
```

## Base de datos

El sistema utiliza PostgreSQL como gestor de base de datos. Se debe asegurar que el servicio esté activo y que exista la base de datos configurada en el archivo .env.

## Variables de entorno

El sistema requiere un archivo .env en el backend con la configuración de conexión a la base de datos:

```
DB_HOST=localhost
DB_PORT=5432
DB_USER=postgres
DB_PASSWORD=1234
DB_NAME=cosecha_red
```

## Ejecución del proyecto

Para iniciar el servidor backend:

```
uvicorn app.main:app --reload
```

## Acceso al sistema

Una vez ejecutado el backend, el sistema estará disponible en:

<http://127.0.0.1:8000>

Y la documentación automática en:

<http://127.0.0.1:8000/docs>

## Variables de entorno

Las variables de entorno permiten almacenar información de configuración del sistema de manera externa al código fuente, facilitando la conexión con servicios como la base de datos y mejorando la seguridad y organización del proyecto. En el sistema “Cosecha Red” se utiliza un archivo `.env` dentro de la carpeta backend para definir los parámetros de conexión utilizados por FastAPI y PostgreSQL.

El archivo `.env` contiene la siguiente configuración:

```
DB_HOST=localhost
DB_PORT=5432
DB_USER=postgres
DB_PASSWORD=1234
DB_NAME=cosecha_red
```

| Variable    | Descripción                                         |
|-------------|-----------------------------------------------------|
| DB_HOST     | Dirección del servidor de base de datos             |
| DB_PORT     | Puerto utilizado por PostgreSQL                     |
| DB_USER     | Usuario de acceso a PostgreSQL                      |
| DB_PASSWORD | Contraseña del usuario de base de datos             |
| DB_NAME     | Nombre de la base de datos utilizada por el sistema |

Las variables de entorno son cargadas automáticamente mediante la librería `python-dotenv`, permitiendo que el sistema acceda a la configuración sin necesidad de escribir datos sensibles directamente en el código fuente.



## Control de versiones / Gitflow

El proyecto “Cosecha Red” utiliza Git y GitHub como herramientas de control de versiones, permitiendo administrar los cambios realizados en el código fuente, mantener un historial de modificaciones y facilitar el trabajo colaborativo entre los integrantes del equipo.

Para organizar el desarrollo se utiliza una estrategia basada en Gitflow, donde cada rama cumple una función específica dentro del ciclo de trabajo del proyecto. Esto permite separar el código estable, el desarrollo activo y las nuevas funcionalidades.

La rama main se utiliza para mantener la versión estable del sistema. La rama develop funciona como rama principal de integración, donde se unen los avances realizados por el equipo antes de preparar una versión final.

Para el desarrollo de nuevas funcionalidades se utilizan ramas feature, creadas a partir de develop. Una vez finalizado el trabajo, los cambios se integran mediante Pull Request para su revisión antes de ser incorporados al proyecto. Este flujo permite mantener un mejor control del desarrollo, reducir conflictos entre integrantes y asegurar que los cambios sean revisados antes de formar parte de la versión principal del sistema.

## Convención de ramas reales

El proyecto “Cosecha Red” utiliza una convención de nombres para las ramas de Git con el objetivo de mantener un flujo de trabajo organizado y facilitar la identificación de funcionalidades, correcciones y versiones del sistema.

La rama principal main se utiliza para almacenar la versión estable del proyecto, mientras que la rama develop contiene la integración de los cambios en desarrollo antes de ser liberados oficialmente.

Para nuevas funcionalidades se utilizan ramas con el prefijo feature/, permitiendo trabajar de forma independiente sin afectar el código principal del sistema.

```
main
develop
feature/login
feature/register
feature/catalogo
feature/publicaciones
release/v1.0.0
hotfix/error-login
```

Las ramas release son utilizadas para preparar versiones oficiales del sistema antes de ser publicadas, mientras que las ramas hotfix permiten corregir errores críticos detectados en producción. Esta convención facilita el trabajo colaborativo, mejora la organización del proyecto y permite mantener un control adecuado sobre las diferentes etapas del desarrollo.

## Tags y Releases

El proyecto “Cosecha Red” utiliza tags y releases en GitHub para identificar y administrar las diferentes versiones del sistema durante el proceso de desarrollo. Los tags permiten marcar puntos importantes dentro del historial del proyecto, especialmente versiones estables listas para ser entregadas o publicadas. Para este proyecto se utiliza una nomenclatura basada en versionado semántico (SemVer), siguiendo el formato:

vMAJOR.MINOR.PATCH

En la primera versión funcional del sistema se utiliza el tag:

v1.0.0

Donde:

- MAJOR representa cambios grandes o incompatibles.
- MINOR representa nuevas funcionalidades.
- PATCH representa correcciones menores o ajustes.

Las releases se utilizan para agrupar oficialmente una versión del sistema junto con sus funcionalidades implementadas, correcciones realizadas y documentación correspondiente.

Para crear un tag en Git se utiliza:

```
git tag -a v1.0.0 -m "Release inicial del sistema Cosecha Red"
```

Y para subirlo al repositorio remoto:

```
git push origin v1.0.0
```

El uso de tags y releases permite mantener un mejor control de versiones, facilitar futuras actualizaciones y conservar un registro organizado de la evolución del proyecto.

## Plan de pruebas

El plan de pruebas del sistema “Cosecha Red” tiene como objetivo verificar que las funcionalidades principales de la plataforma trabajen correctamente y cumplan con los requisitos definidos para la versión v1.0.0. Las pruebas se enfocan en validar el registro de usuarios, inicio de sesión, control de roles, conexión con la base de datos, funcionamiento de la API y correcta respuesta del sistema ante datos válidos e inválidos.

Las pruebas se realizarán de forma manual y funcional, utilizando el entorno local de desarrollo. Para la validación de endpoints se podrá utilizar Swagger UI, disponible desde FastAPI, y herramientas como Postman cuando sea necesario.

Tipos de pruebas consideradas:

- Pruebas funcionales: verifican que cada funcionalidad del sistema cumpla con lo esperado.
- Pruebas de integración: validan la comunicación entre frontend, backend y base de datos.
- Pruebas de autenticación: comprueban el registro, inicio de sesión y uso de token JWT.
- Pruebas de validación: verifican que el sistema rechace datos incompletos o incorrectos.
- Pruebas de interfaz: revisan que las pantallas principales sean comprensibles y permitan la interacción del usuario.

El resultado de cada prueba debe registrarse indicando el caso probado, los pasos ejecutados, el resultado esperado, el resultado obtenido y su estado final como aprobado o fallido.



## Casos de prueba

Los casos de prueba permiten verificar que las funcionalidades principales del sistema “Cosecha Red” trabajen correctamente según los requisitos definidos durante el análisis y desarrollo del proyecto. Cada caso de prueba describe una acción específica realizada dentro del sistema, indicando los pasos necesarios, el resultado esperado y el estado obtenido después de ejecutar la prueba. Los casos de prueba fueron diseñados para validar procesos importantes como registro de usuarios, autenticación, validación de datos y funcionamiento de la API REST.

| Casos de prueba realizados |               |                                    |                                    |          |
|----------------------------|---------------|------------------------------------|------------------------------------|----------|
| ID                         | Módulo        | Caso de prueba                     | Resultado esperado                 | Estado   |
| CP-01                      | Registro      | Registrar usuario válido           | Usuario creado correctamente       | Aprobado |
| CP-02                      | Registro      | Registrar usuario con datos vacíos | Mostrar mensaje de error           | Aprobado |
| CP-03                      | Login         | Inicio de sesión válido            | Generar token JWT                  | Aprobado |
| CP-04                      | Login         | Inicio de sesión inválido          | Mostrar credenciales incorrectas   | Aprobado |
| CP-05                      | API           | Consultar endpoint protegido       | Retornar información correctamente | Aprobado |
| CP-06                      | Validación    | Acceso sin token                   | Retornar error 401 Unauthorized    | Aprobado |
| CP-07                      | Base de datos | Conexión con PostgreSQL            | Conexión establecida correctamente | Aprobado |

### Evidencias de pruebas

Las evidencias de pruebas corresponden a los resultados obtenidos durante la validación del sistema “Cosecha Red”. Estas evidencias permiten demostrar el correcto funcionamiento de las principales funcionalidades implementadas en el proyecto. Las evidencias incluyen capturas de pantalla del entorno de desarrollo, ejecución del backend, conexión con la base de datos, pruebas realizadas desde Swagger UI y respuestas obtenidas desde la API REST.

Entre las principales evidencias presentadas se encuentran:

- Ejecución correcta del servidor FastAPI.
- Conexión exitosa con PostgreSQL.
- Registro de usuarios desde la API.
- Inicio de sesión y generación de token JWT.
- Validación de acceso a endpoints protegidos.
- Respuestas JSON obtenidas desde Swagger UI.
- Validaciones de errores y códigos HTTP.
- Funcionamiento general del sistema desde el entorno local.

Las capturas incluidas permiten respaldar técnicamente el funcionamiento de la versión v1.0.0 del sistema y sirven como evidencia del proceso de pruebas realizado durante el desarrollo.